# Natural Language Processing

Christophe.rodrigues@devinci.fr

# Summary

- Course Modalities
- Definitions
- Classic Libraries
- Modern Libraries

# Course Modalities

- 3 evaluations during this course:
    - Unsupervised project (40%)
    - Supervised project (40%)
    - Final Quiz (20%)

# Definition (according to wikipedia)

**Natural language processing** (**NLP**) is the processing of natural language information by a computer. NLP is a subfield of computer science and is closely associated with artificial intelligence. NLP is also related to information retrieval, knowledge representation, computational linguistics, and linguistics more broadly.[1]

Major processing tasks in an NLP system include: speech recognition, text classification, natural language understanding, and natural language generation.

# Text in quantity

- In 2025 :

  - 16 million texts are sent every minute

  - 156 million emails are sent

  - Of those emails, 103.5 million are spam

  - The Google search engine alone processes more than 40,000 searches every second (3.5 billion searches per day).

  - Internet searches worldwide have reached 5 billion searches a day.

# Language in quality

- To share information, to communicate with others
- To learn
- To store information

Language shape the way we think

# Automatic Translation Example

Linguee

français ↔ anglais

à â é è ê ë ï î ô ù û ç œ æ

boot

| | |
|---|---|
| I want it to start at boot time. openbsd.gr | Je souhaite que ce service soit lancé au démarrage. openbsd.gr |
| The fact that this minister, who boasts about recognizing the Quebec nation, does not even deign to speak French-which is his first language, to boot-when giving a speech in public, shows a flagrant lack of respect for the Francophonie. www2.parl.gc.ca | Le fait que ce ministre, qui se targue de reconnaître la nation québécoise, ne daigne même pas utiliser le français - par surcroît sa propre langue maternelle - pour prononcer un discours en public démontre un manque flagrant de respect envers la Francophonie. www2.parl.gc.ca |
| [...] own time, suited up and stood by the doors prior to security screening and collected the funds in their firefighter boot. ottawa-airport.com | [...] travail, pendant leurs heures libres, pour recueillir des fonds dans leur botte de pompier à la porte du Contrôle de sécurité. ottawa-airport.com |
| If people have a chance to reduce their oil heating costs by replacing their windows and get a tax credit to boot, they will do home renovations. www2.parl.gc.ca | [...] au pétrole en changeant les fenêtres et en ayant un crédit d'impôt de surcroît, ils feront les rénovations domiciliaires. www2.parl.gc.ca |
| This does not mean that booting is done from the first device in this list; it only means that the first attempt to boot an operating system will be from this device. ftp.thecus.com | Cela ne veut pas dire que le démarrage est effectué à partir du premier périphérique dans cette liste ; cela signifie uniquement que la première tentative de démarrage d'un système d'exploitation sera faite à partir de ce périphérique. ftp.thecus.com |
| The rear of the boot is built up to shore up the ankle, which is constantly under pressure in the free technique. sochi2014.com | L'arrière des chaussures comporte une tige haute qui soutient la cheville, sur laquelle une pression constante est exercée en style libre. sochi2014.com |

DeepL

anglais français

AI for languages

L'IA pour les langues

Autres tradu... Diction...

Autre

L'IA a

L'inte es

au service

appliquée

dans les langues

mise

des langues

appliqué

service

et les langues

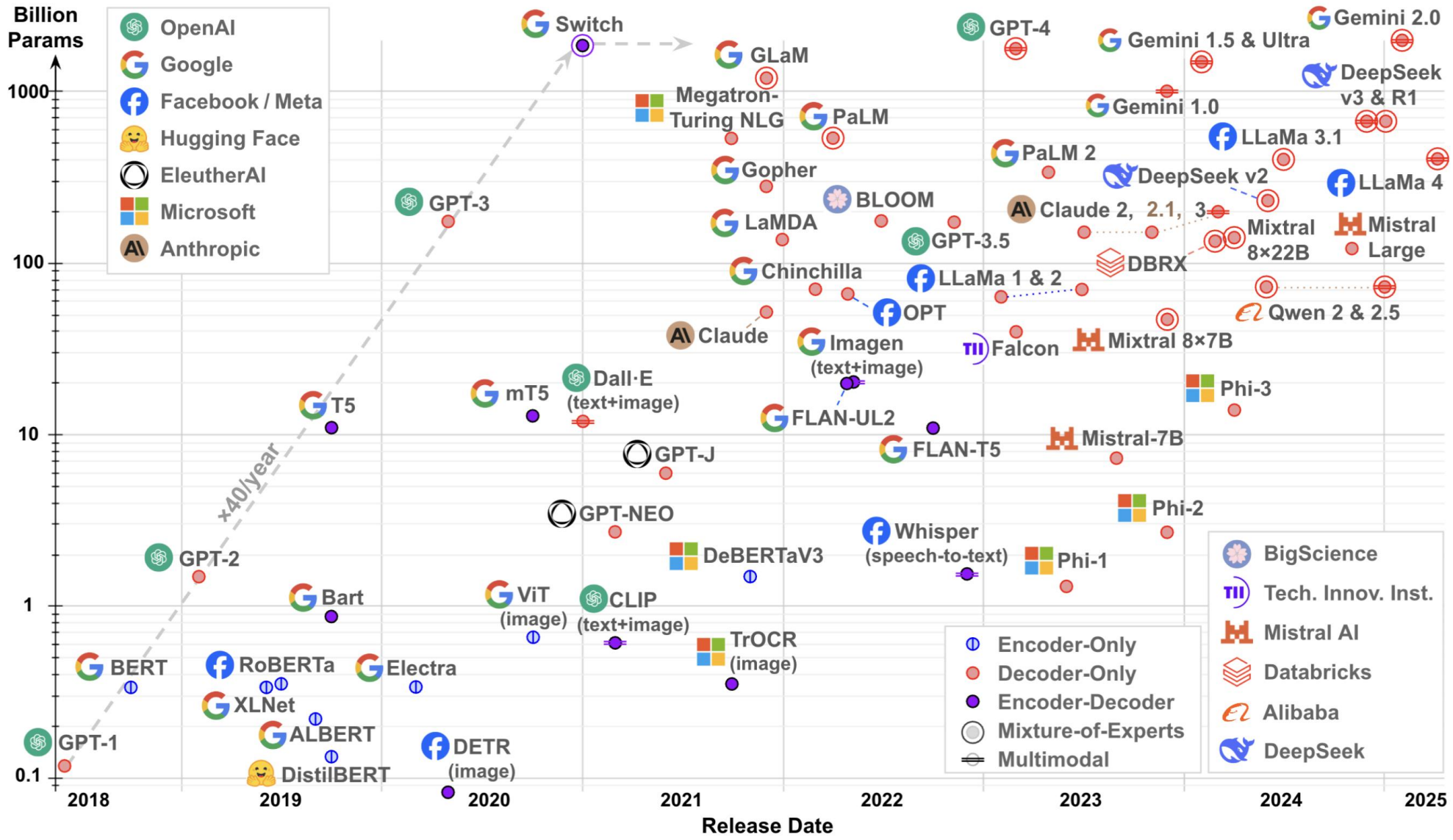
# NLP Difficulty

- Lot of problems in NLP remain difficult even today.
- But one specificity of NLP is that lot of problems are not just difficult to find a solution but also to evaluate the solution.
- For example : Finding a good summary to a given document is difficult, but evaluating the quality of summary is also difficult.
- Because the process of realisation and evaluation are in reality very similar.

# Symbolic versus Statistical history

- From 1950 to 1990 : Symbolic approaches
- From 1990 to Now : Statistical approaches
  - In 2010 first breakthrough with Neural Networks : Word2Vec
  - In 2017 second breakthrough with Transformers : BERT
  - In 2022 public release of ChatGPT

# Evolution of Transformer based models



# Some definitions

- N-gram = subsenquence of n items
- The items can be phonemes, syllables, letters, words
- Corpus (plural corpora) = a computer-readable collection of text or speech
- Lemma = a set of lexical forms having the same stem, the same major part-of-speech, and the same word sense
- Types = number of distinct words in a corpus
- Vocabulary = set of all distinct tokens of a corpus
- Tokens = (not always) the words

- Bag of words = set of words (without sequence order)
- Lemmatization = reducing a word to the root

  Example : *am*, *are* and *is* have the shared lemma be

Stemming = cutting suffixe of words

Example :

  – ATIONAL → ATE (relational → relate)

  – ING → _ (doing → do)

  – SSES → SS (grasses → grass)

Stemming and Lemmatization are useful to reduce vocabulary entropy

# Representing Text on a Computer

- We need to represent characters in a binary world
- One of the first attempt in 1960: ASCII (*American Standard Code for Information Interchange)*
- 128 codes on 7 bits
- 95 printable characters (enough for english but not for others languages)

!"#$%&'()*+,-./

0123456789:;<=>?

@ABCDEFGHIJKLMNO

PQRSTUVWXYZ[\]^_

`abcdefghijklmno

pqrstuvwxyz{|}~

# ASCII TABLE

| Decimal | Hex | Char | Decimal | Hex | Char | Decimal | Hex | Char | Decimal | Hex | Char |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | [NULL] | 32 | 20 | [SPACE] | 64 | 40 | @ | 96 | 60 | ` |
| 1 | 1 | [START OF HEADING] | 33 | 21 | ! | 65 | 41 | A | 97 | 61 | a |
| 2 | 2 | [START OF TEXT] | 34 | 22 | " | 66 | 42 | B | 98 | 62 | b |
| 3 | 3 | [END OF TEXT] | 35 | 23 | # | 67 | 43 | C | 99 | 63 | c |
| 4 | 4 | [END OF TRANSMISSION] | 36 | 24 | $ | 68 | 44 | D | 100 | 64 | d |
| 5 | 5 | [ENQUIRY] | 37 | 25 | % | 69 | 45 | E | 101 | 65 | e |
| 6 | 6 | [ACKNOWLEDGE] | 38 | 26 | & | 70 | 46 | F | 102 | 66 | f |
| 7 | 7 | [BELL] | 39 | 27 | ' | 71 | 47 | G | 103 | 67 | g |
| 8 | 8 | [BACKSPACE] | 40 | 28 | ( | 72 | 48 | H | 104 | 68 | h |
| 9 | 9 | [HORIZONTAL TAB] | 41 | 29 | ) | 73 | 49 | I | 105 | 69 | i |
| 10 | A | [LINE FEED] | 42 | 2A | * | 74 | 4A | J | 106 | 6A | j |
| 11 | B | [VERTICAL TAB] | 43 | 2B | + | 75 | 4B | K | 107 | 6B | k |
| 12 | C | [FORM FEED] | 44 | 2C | , | 76 | 4C | L | 108 | 6C | l |
| 13 | D | [CARRIAGE RETURN] | 45 | 2D | - | 77 | 4D | M | 109 | 6D | m |
| 14 | E | [SHIFT OUT] | 46 | 2E | . | 78 | 4E | N | 110 | 6E | n |
| 15 | F | [SHIFT IN] | 47 | 2F | / | 79 | 4F | O | 111 | 6F | o |
| 16 | 10 | [DATA LINK ESCAPE] | 48 | 30 | 0 | 80 | 50 | P | 112 | 70 | p |
| 17 | 11 | [DEVICE CONTROL 1] | 49 | 31 | 1 | 81 | 51 | Q | 113 | 71 | q |
| 18 | 12 | [DEVICE CONTROL 2] | 50 | 32 | 2 | 82 | 52 | R | 114 | 72 | r |
| 19 | 13 | [DEVICE CONTROL 3] | 51 | 33 | 3 | 83 | 53 | S | 115 | 73 | s |
| 20 | 14 | [DEVICE CONTROL 4] | 52 | 34 | 4 | 84 | 54 | T | 116 | 74 | t |
| 21 | 15 | [NEGATIVE ACKNOWLEDGE] | 53 | 35 | 5 | 85 | 55 | U | 117 | 75 | u |
| 22 | 16 | [SYNCHRONOUS IDLE] | 54 | 36 | 6 | 86 | 56 | V | 118 | 76 | v |
| 23 | 17 | [END OF TRANS. BLOCK] | 55 | 37 | 7 | 87 | 57 | W | 119 | 77 | w |
| 24 | 18 | [CANCEL] | 56 | 38 | 8 | 88 | 58 | X | 120 | 78 | x |
| 25 | 19 | [END OF MEDIUM] | 57 | 39 | 9 | 89 | 59 | Y | 121 | 79 | y |
| 26 | 1A | [SUBSTITUTE] | 58 | 3A | : | 90 | 5A | Z | 122 | 7A | z |
| 27 | 1B | [ESCAPE] | 59 | 3B | ; | 91 | 5B | [ | 123 | 7B | { |
| 28 | 1C | [FILE SEPARATOR] | 60 | 3C | < | 92 | 5C | \ | 124 | 7C | \| |
| 29 | 1D | [GROUP SEPARATOR] | 61 | 3D | = | 93 | 5D | ] | 125 | 7D | } |
| 30 | 1E | [RECORD SEPARATOR] | 62 | 3E | > | 94 | 5E | ^ | 126 | 7E | ~ |
| 31 | 1F | [UNIT SEPARATOR] | 63 | 3F | ? | 95 | 5F | _ | 127 | 7F | [DEL] |

# Nowadays : UTF-8

- UTF for Unicode Transformation format (1992)
- Variable width character encoding, Between 1 and 4 octets

  → 1,112,064 possible codes

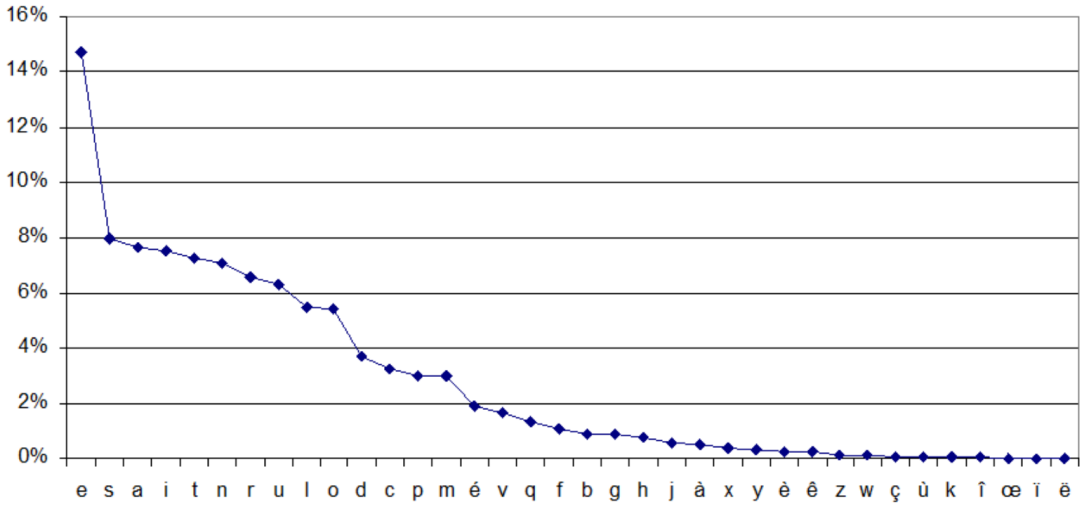
UTF-8 is the most used encoding standard on internet

In 2026, 99% of webpages are transmitted as UTF-8

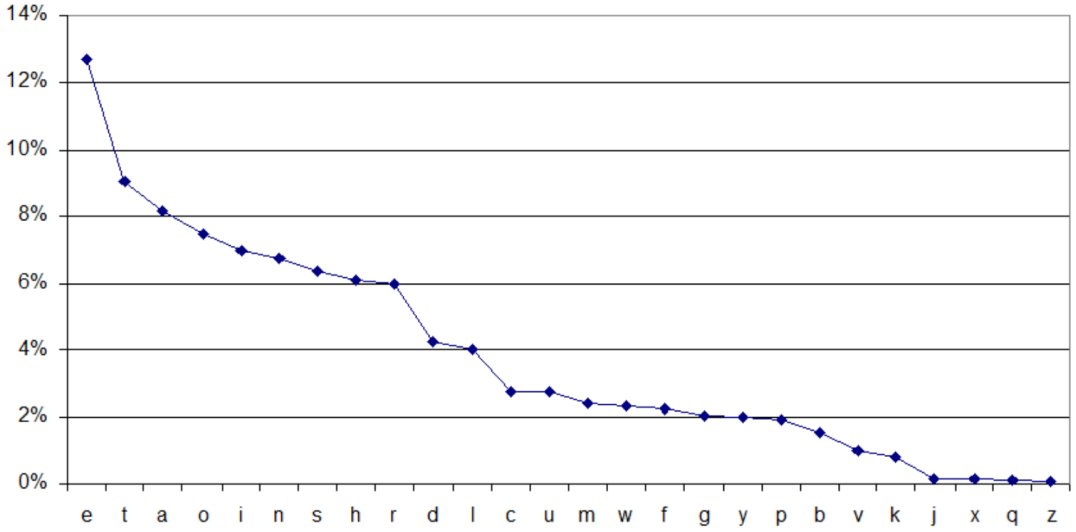
# Character Distribution

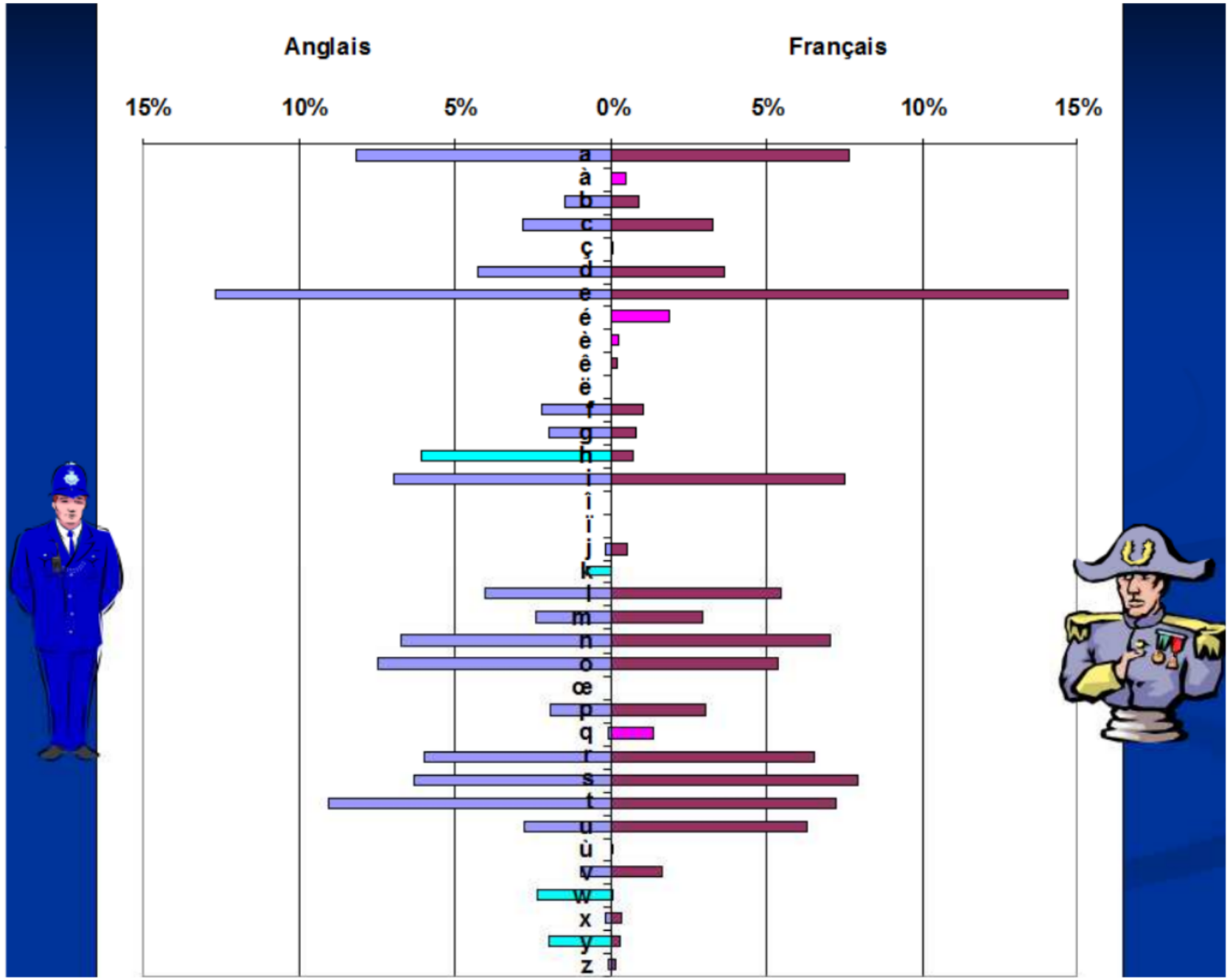
Unigrams

French character frequency




English character frequency




Anglais
Français
15%
10%
5%
0%
5%
10%
15%
a
à
b
c
ç
d
e
é
è
ê
ë
f
g
h
i
î
ï
j
k
l
m
n
o
œ
p
q
r
s
t
u
ù
v
w
x
y
z

# Bigrams

| Français | Anglais | Allemand | Italien | Espagnol | Portugais |
|---|---|---|---|---|---|
| on | th | en | di | de | de |
| es | on | er | on | en | es |
| de | an | ch | ri | er | to |
| te | he | ei | er | on | da |
| nt | er | un | al | ci | os |
| re | nd | de | to | es | re |
| en | in | nd | ta | re | en |
| le | ti | ge | ne | os | er |
| it | al | re | in | io | te |
| er | re | in | re | la | ra |
| et | io | ie | it | ra | nt |
| ti | en | te | io | na | em |
| ou | ri | ng | de | ec | do |
| io | of | he | li | al | di |
| la | or | ne | en | ad | it |
| oi | at | ht | ni | da | al |
| ne | it | ic | tt | to | ad |
| me | to | be | la | nt | co |
| ro | ed | it | ll | ie | ei |
| ns | nt | sc | el | el | as |

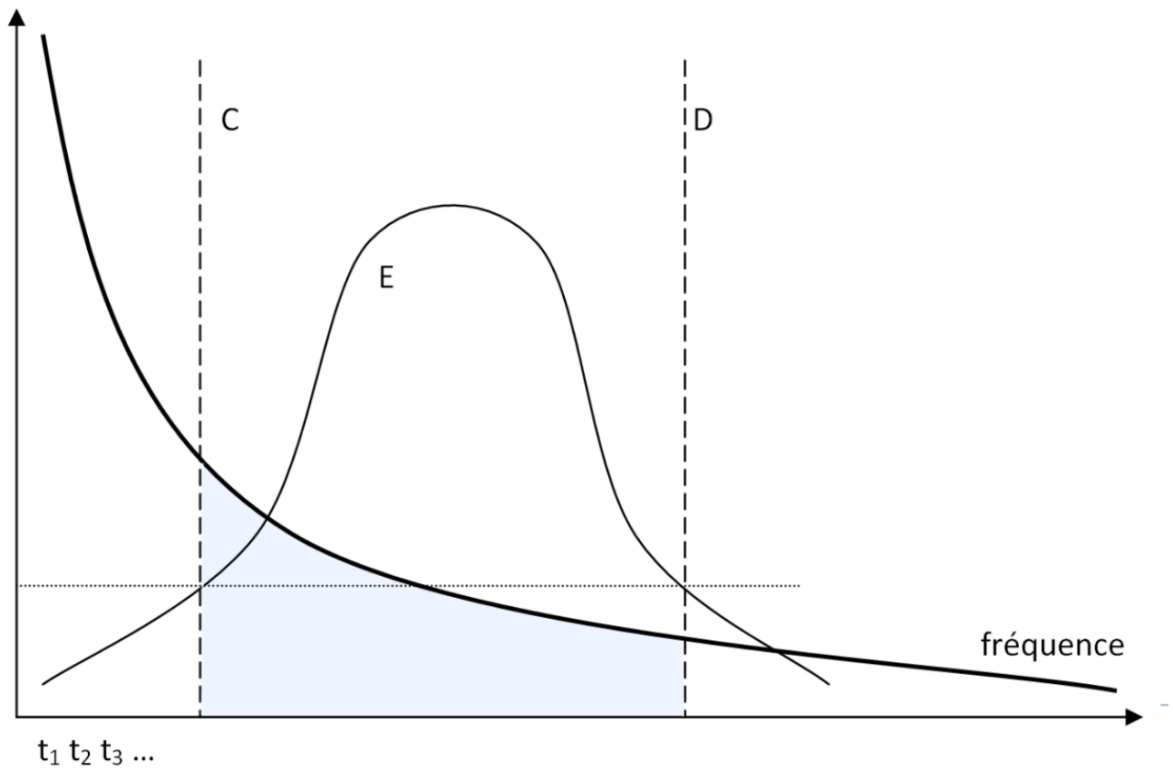
Trigrams

| Français | Anglais | Allemand | Italien | Espagnol | Portugais |
|---|---|---|---|---|---|
| ion | the | der | ion | ion | ent |
| tio | and | und | zio | cio | ito |
| ent | ion | ein | ell | rec | eit |
| oit | tio | ung | one | ere | dir |
| ati | ati | cht | lla | der | ire |
| roi | igh | ich | rit | ien | rei |
| dro | ght | sch | itt | cho | ção |
| men | rig | che | del | ent | ade |
| tou | ent | ech | iri | ech | dad |
| con | ver | die | dir | aci | men |
| res | one | rec | ess | ona | nte |
| que | all | ine | ent | nte | dos |
| les | eve | eit | azi | con | ess |
| des | ery | gen | tto | ene | con |
| eme | his | ver | ere | tod | tod |

# Distribution of words : The Zipf’s Law

- the frequency of any word is inversely proportional to its rank in the frequency table
- In Ulysse from James Joyce :
  - the most frequent word occurs 8000 times
  - the tenth, 800 times
  - the hundredth, 80 times
  - the tousandth, 8 times.

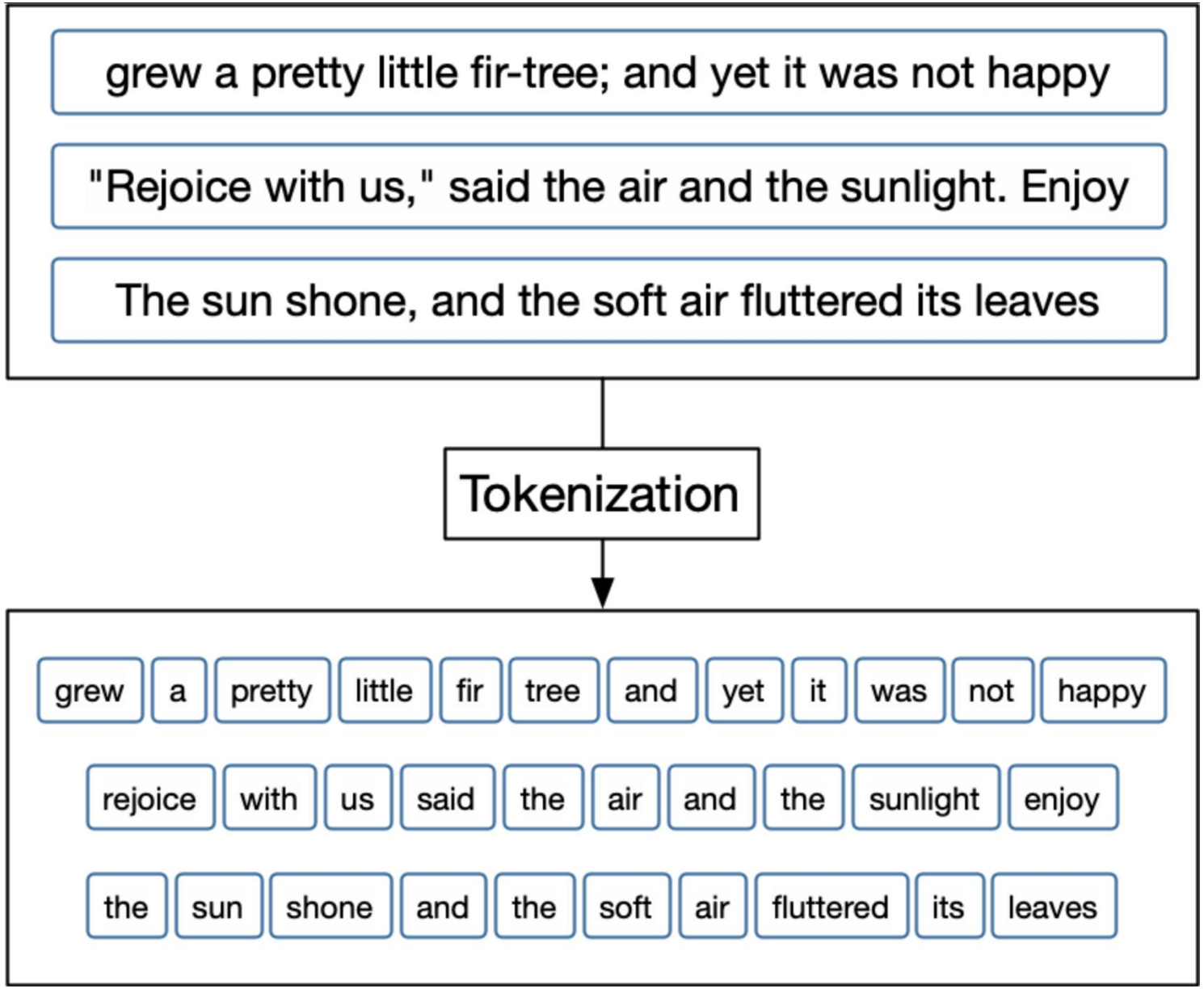
# Distribution of words : The Zipf’s Law




Most frequents and rare words are less informatives

# Tokenization

Converting a
text into tokens



# Tokenization – How to split?

**Tokenization on rules**

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| Let | ‘s | go’ | for | a | walk | tonight | ! |

**Tokenization on Punctuation**

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| Let | ’ | s | go | ’ | for | a | walk | tonight | ! |

**Tokenization on white spaces**

| | | | | | |
|---|---|---|---|---|---|
| Let’s | go’ | for | a | walk | tonight! |

# Tokenization, the early stages

- With Regular expressions
  - [a-z] any letter between a and z
  - [0-9] any number between 0 and 9
  - | for alternatives
    - relat(ional|e) covers relational and relate

Useful also to find complex strings

* : last character is repeated 0,1 or more times

ex : ab* covers : ab, abb, abbb, abbbb...

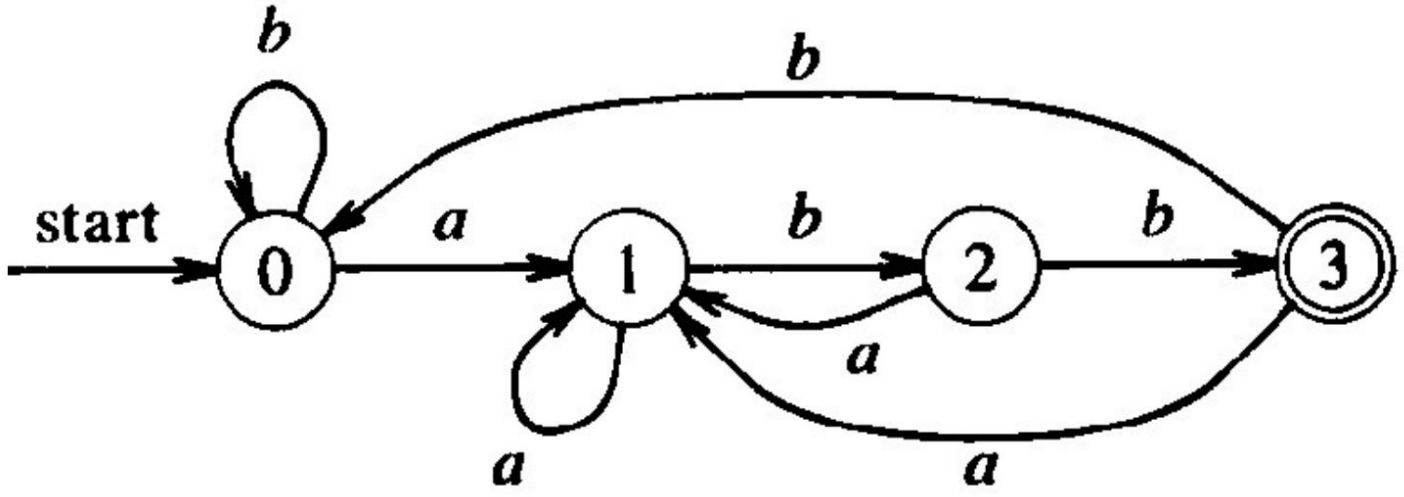
a(bc)* covers : a, abc, abcbc, abcbcbc...

?: last character appeared 0 or 1 time.

ex :plurals? Covers plural and plurals

# Regular Expressions are Deterministic Finite Automaton

To test and develop regex :https://regex101.com/
^\+?[0-9]{2}-?([0-9]{9}|[0-9]-[0-9]{2}-[0-9]{2}-[0-9]{2} -[0-9]{2})$




abb, aaabaabbabb are covered by the automaton

# Subword Tokenization : Byte-Pair Encoding

- Byte-Pair Encoding(2016)
- To define a vocabulary : learn how to split words according to a corpus

# Byte-Pair Encoding(BPE)

- reduces out-of-vocabulary issues,
- balances vocabulary size and contextual expressiveness,
- preserves frequent whole words while breaking rare words into meaningful parts. Control the exact vocabulary size

- BPE Tokenizer is used by Open AI in GPT, GPT-2, but also in several other transformer based models like LlaMA, RoBERTa, BART, DeBERTa...

# BPE Example

- Add a end of word character ‘·’
- Start with all tokenized words and their frequency:
- Example corpus : «  low low low low low lowest lowest newer newer newer newer newer newer wider wider wider new new »

corpus representation:
5x l o w ·
2x l o w e s t ·
6x n e w e r ·
3x w i d e r ·
2x n e w ·

Vocabulary :
·,d,e,i,l,n,o,r,s,t,w

# BPE algorithm

- 0) vocabulary is all possible characters {a,b,...}
- Repeat :
  1) Find the most frequent adjacents tokens (Ex: ‘a’, ‘n’)
  2) Concate and Add to the vocabulary (Ex: ‘an’)
  3) Replace occurrences of step 1) by concatenated one of step 2)

- Until : #k merges made

*corpus*:
5x l o w ·
2x l o w e s t ·
6x n e w **<u>e r</u>** ·
3x w i d **<u>e r</u>** ·
2x n e w ·

*Vocabulary :*
·,d,e,i,l,n,o,r,s,t,w

Merge **e r** into **er**, and replace
*corpus:*
5x l o w ·
2x l o w e s t ·
6x n e w er ·
3x w i d er ·
2x n e w ·

*Vocabulary :*
·,d,e,i,l,n,o,r,s,t,w,er

*corpus*:
5x l o w ·
2x l o w e s t ·
6x n e w **er ·**
3x w i d **er ·**
2x n e w ·

*Vocabulary :*
·,d,e,i,l,n,o,r,s,t,w,er

Merge **er** · into **er**·, and replace
*corpus:*
5x l o w ·
2x l o w e s t ·
6x n e w er·
3x w i d er·
2x n e w ·

*Vocabulary :*
·,d,e,i,l,n,o,r,s,t,w,er,er·

*corpus*:
5x l o w ·
2x l o w e s t ·
6x **<u>n e</u>** w er·
3x w i d er·
2x **<u>n e</u>** w ·

*Vocabulary :*
·,d,e,i,l,n,o,r,s,t,w,er,er·

Merge **n e** into ne , and replace
*corpus:*
5x l o w ·
2x l o w e s t ·
6x ne w er·
3x w i d er·
2x ne w ·

*Vocabulary :*
·,d,e,i,l,n,o,r,s,t,w,er,er·,ne

| Merges | Current Vocabulary |
| --- | --- |
| (ne,w) | ·,d,e,i,l,n,o,r,s,t,w,er,er·,ne,new |
| (l,o) | ·,d,e,i,l,n,o,r,s,t,w,er,er·,ne,new,lo |
| (lo,w) | ·,d,e,i,l,n,o,r,s,t,w,er,er·,ne,new,lo,low |
| (new,er·) | ·,d,e,i,l,n,o,r,s,t,w,er,er·,ne,new,lo,low,newer· |
| (low, ·) | ·,d,e,i,l,n,o,r,s,t,w,er,er·,ne,new,lo,low,newer·,low· |

# Once learned, vocabulary can tokenize the corpus used for our application

- On the test corpus, apply each merge greedily in the learned order (no frequency involved)
- V = ·,d,e,i,l,n,o,r,s,t,w,er,er·,ne,new,lo,low,newer·,low·
- Test corpus :
  - « n e w e r · » tokenized as « newer· »
  - « l o w e r · » tokenized as « low » and «er· »

    (word lower doesn’t appear at learning time, it’s an out of vocabulary(oov))

In conclusion we keep only frequent words and frequent subwords as tokens

# WordPiece

- Tokenizer used in BERT family models like DistilBERT, MobileBERT, Funnel Transformers, MPNET...

- Similar to BPE tokenizer, but :
  - ***[UNK]*** for unknown tokens
  - ***## prefix*** for subwords that don't start a word
  - Each merge of A and B tokens are scored :

    *Score(A, B) = (freq(AB) * len(vocab)) / (freq(A) * freq(B))*

    *priority to the merged tokens where individual parts are less frequent*

# Example of my final project as student on Master 2 on my NLP course

Sentiment analysis project in 2008:

- Encoding loading of text (converting to iso-8859-1) (C++)
- Sentence Tokenization (Lisp)
- Word Tokenization (Lisp)
- SentiwordNet Dataset loading (Python)
- Data preparation and Supervised Learning model (C/C++)

# With NLTK library (Python)

NLTK for **N**atural **L**anguage **T**ool **K**it

Example code:

```
from nltk.corpus import sentiwordnet as swn
Sentence = ‘this is a sentence example’
tokenSentence = nltk.word_tokenize(sentence)
breakdown = swn.senti_synset('breakdown.n.01’)
Print(breakdown)
```

<breakdown.n.03: PosScore=0.0 NegScore=0.25>

# With Hugging Face

```
from transformers import pipeline
pipe = pipeline("text-classification")
pipe("This restaurant is awesome")
        [{'label': 'POSITIVE', 'score': 0.9998743534088135}]
```

# Hugging Face (Transformers for all)

- The company was founded in 2016 by French entrepreneurs Clément Delangue, Julien Chaumond, and Thomas Wolf
- The main contribution is the transformers library
- Hugging Face democratize and ease the utilization of transformers
- its platform allows users to share machine learning models and datasets and showcase their work.

# Hugging Face Pipeline Applications

- audio-classification
- automatic-speech-recognition
- depth-estimation
- document-question-answering
- feature-extraction
- fill-mask
- image-classification
- image-feature-extraction
- image-segmentation
- image-text-to-text
- image-to-image
- image-to-text
- keypoint-matching
- mask-generation
- object-detection
- question-answering
- summarization
- table-question-answering
- text2text-generation
- text-classification (alias sentiment-analysis)
- text-generation
- text-to-audio (alias text-to-speech)
- token-classification (alias "ner")
- translation
- translation_xx_to_yy
- video-classification
- visual-question-answering
- zero-shot-classification
- zero-shot-image-classification
- zero-shot-audio-classification
- zero-shot-object-detection

# Representing Words as vectors

- One-hot vectors

$$w^{aardvark} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, w^{a} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, w^{at} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}, \cdots \; w^{zebra} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix}$$

$$(w^{hotel})^T w^{motel} = (w^{hotel})^T w^{cat} = 0$$

# Word vectors

- Comparing two sentences can be resume to compute a scalar product between two vectors.
- In order to reduce vectors size :
  - keep only the n most frequent tokens
  - normalize the most frequents tokens according to zipf’s law.

# The rare linguistic hypothesis that drives modern models

"You shall know a word by the company it keeps" (Firth, 1957)

Distributional Hypothesis : "Words that occur in the same contexts tend to have similar meanings" (Harris, 1954)

# Classic vector representation (before 2013)

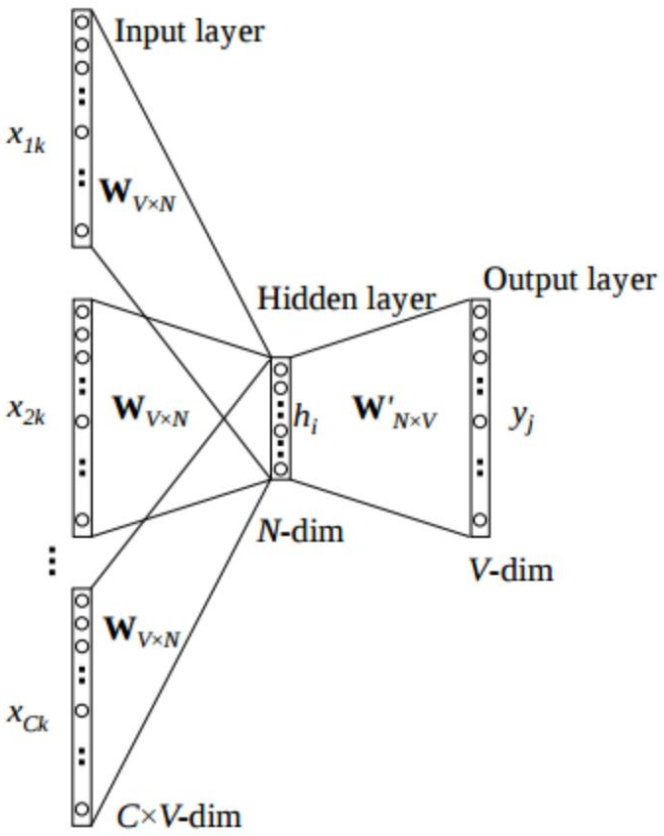
Word co-occurrence  matrix + Dimensional Reduction (SVD)

| Words\C# | $w_1$ | $w_2$ | $w_3$ | ... | $w_n$ |
|---|---|---|---|---|---|
| $w_1$ | $C(w_1, w_1)$ | $C(w_1, w_2)$ | $C(w_1, w_3)$ | ... | $C(w_1, w_n)$ |
| $w_2$ | $C(w_2, w_1)$ | $C(w_2, w_2)$ | $C(w_2, w_3)$ | ... | $C(w_2, w_n)$ |
| $w_3$ | $C(w_3, w_1)$ | $C(w_3, w_2)$ | $C(w_3, w_3)$ | ... | $C(w_3, w_n)$ |
| ... | ... | ... | ... | ... | ... |
| $w_n$ | $C(w_n, w_1)$ | $C(w_n, w_2)$ | $C(w_n, w_3)$ | ... | $C(w_n, w_n)$ |

# Linguistic Regularities in Continuous Space Word Representations

2013

**Tomas Mikolov*, Wen-tau Yih, Geoffrey Zweig**
Microsoft Research
Redmond, WA 98052




Continous Bag of Words Models (CBOW) aka **Word2Vec**

Goal : predict the probability of a word given context words.
Example : "The cat jumped over the puddle."

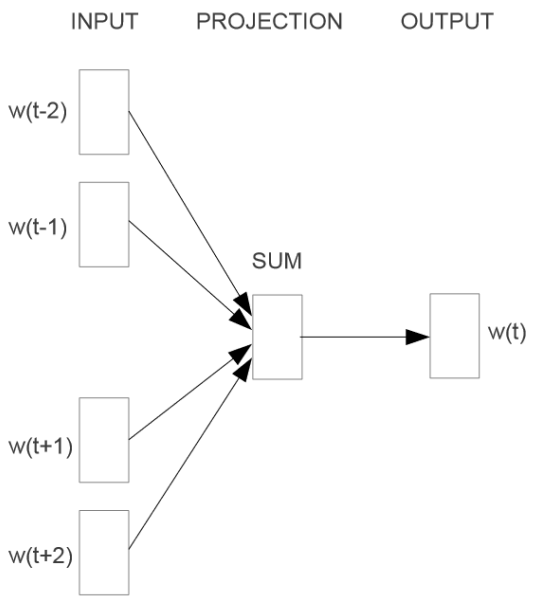
for the center word "jumped", the context is :
{"The", "cat", 'over", "the', "puddle"}

# Linguistic Regularities in Continuous Space Word Representations

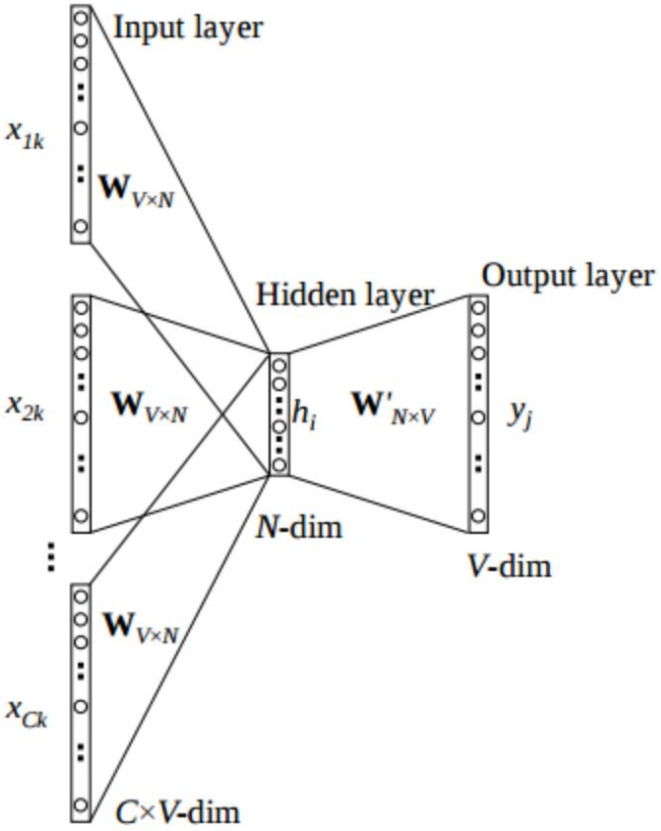
2013

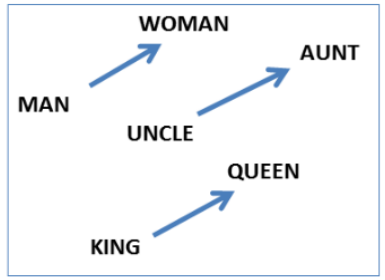
**Tomas Mikolov*, Wen-tau Yih, Geoffrey Zweig**
Microsoft Research
Redmond, WA 98052

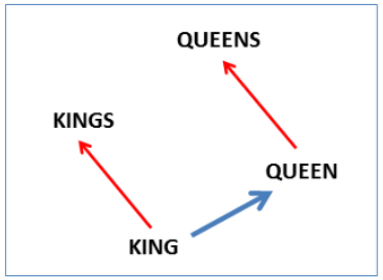
Word2Vec Online Visualization
https://projector.tensorflow.org/




CBOW




King – Man + Queen = Woman

Word2Vec model

# *Don't count, predict!* A systematic comparison of context-counting vs. context-predicting semantic vectors

**Marco Baroni** and **Georgiana Dinu** and **Germán Kruszewski**
Center for Mind/Brain Sciences (University of Trento, Italy)
(marco.baroni|georgiana.dinu|german.kruszewski)@unitn.it